Design and Deployment of a Customer Data Processing System on AWS

Maab Osman

31.03.2026

Table of Contents

Introduction 3

Project Plan 3

- Phase 1 Design 3
- Phase 2 Design 4

Implementation 5

- Phase 1 Implementation and Validation 5
  - Resources Created 5
  - Networking Setup 5
  - Storage Configuration 6
  - Security Configuration 7
  - Compute and Database Deployment 8
  - Implementation Steps 9
  - Problems and Solutions 10
  - Validation 10

Enhancements 11

Conclusions 11

Appendix 12

- Appendix A: CLI Commands Used 12
- Appendix B: GitHub Repository 13
- Appendix C: Architecture Diagram 14
- Appendix D: Screenshots 14

References 14

Introduction

This project focuses on the design and deployment of a lightweight customer data processing system on AWS. The goal of the project is to create a small but realistic cloud architecture that supports storing, processing, and managing customer-related data. In Phase 1, the focus is on building the core infrastructure by using EC2, RDS, and S3.

The chosen topic supports my interest in cloud infrastructure and backend-oriented systems. Instead of building only a simple website, I wanted to create a more business-relevant architecture that reflects how real organizations process and store customer data. The project is designed in a way that allows Phase 2 to extend the solution with cloud management and governance features such as monitoring, alerting, logging, and cost optimization.

The main objective of Phase 1 is to deploy the infrastructure, validate connectivity between the services, and document the implementation so that it can be recreated in the AWS Learner Lab environment.

Project Plan

Phase 1 Design

The Phase 1 architecture is designed as a multi-tier cloud environment for customer data processing. The system is deployed inside a Virtual Private Cloud (VPC) with clearly separated public and private subnets.

The EC2 instance is placed in a public subnet and acts as the application and processing layer of the system. It is responsible for interacting with other AWS services and performing validation tasks. The RDS database is deployed in private subnets across multiple Availability Zones using a DB subnet group. This ensures that the database is not publicly accessible and follows secure design principles.

S3 is used as an object storage service for storing raw customer data files and potential exports. The EC2 instance communicates with S3 to upload and retrieve files.

Security groups are configured to restrict access between components. The RDS instance only allows inbound traffic on port 3306 from the EC2 security group, ensuring that only the

application layer can access the database. Internet access to the EC2 instance is enabled through an Internet Gateway and a public route table.

The architecture of the system is illustrated in Figure 1 below.

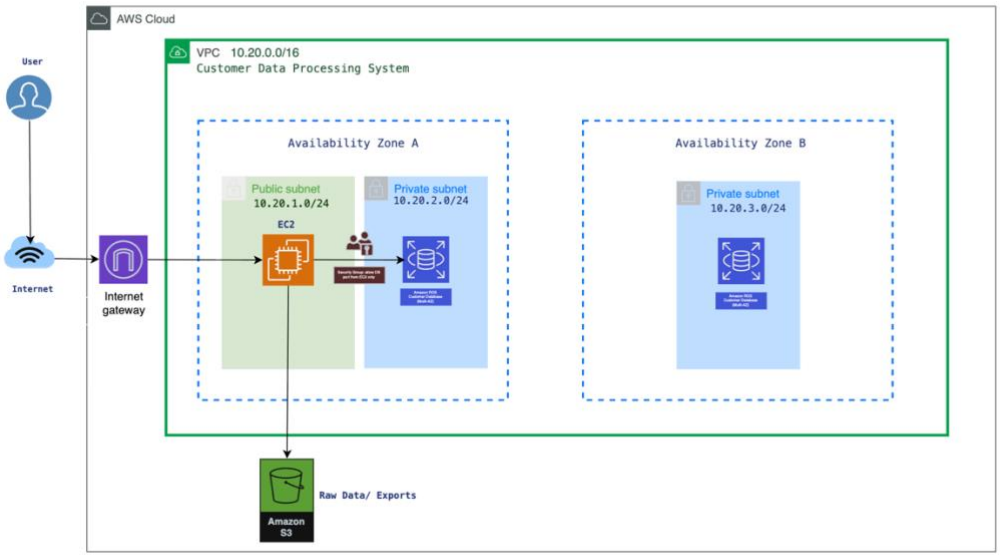

Figure 1: Architecture of the Customer Data Processing System

This architecture was chosen because it is small enough to implement within the limits of the AWS Academy Learner Lab environment, but still realistic enough to reflect a common business use case. It also provides a strong foundation for Phase 2, where governance features such as monitoring, logging, and alerting can be added on top of the deployed environment.

Phase 2 Design

In Phase 2, cloud governance and management features will be added to the existing architecture to improve observability, security, and cost efficiency.

Monitoring will be implemented using Amazon CloudWatch to track EC2 and RDS performance metrics such as CPU utilization and database connections. Alerts will be configured using Amazon SNS to notify when predefined thresholds are exceeded.

Logging will be enabled using AWS CloudTrail to track API activity and system changes.

Additionally, application-level logging can be integrated with CloudWatch Logs.

Security improvements will include the use of IAM roles for EC2 instances to securely access AWS services such as S3 without relying on preconfigured credentials.

Cost optimization strategies will be considered by analyzing resource utilization and ensuring that only necessary services are running.

Implementation

Phase 1 Implementation and Validation

The goal of Phase 1 was to deploy the planned customer data processing architecture in AWS and verify that all components function correctly and can communicate with each other.

Resources Created

The following AWS resources were created:

- Virtual Private Cloud (VPC)
- One public subnet
- Two private subnets in different Availability Zones
- Internet Gateway
- Public route table
- Amazon EC2 instance
- Amazon RDS instance
- DB subnet group
- Amazon S3 bucket
- Security groups for EC2 and RDS

Networking Setup

The networking components were created, including the VPC, subnets, Internet Gateway, and route table. The public subnet was configured with internet access.

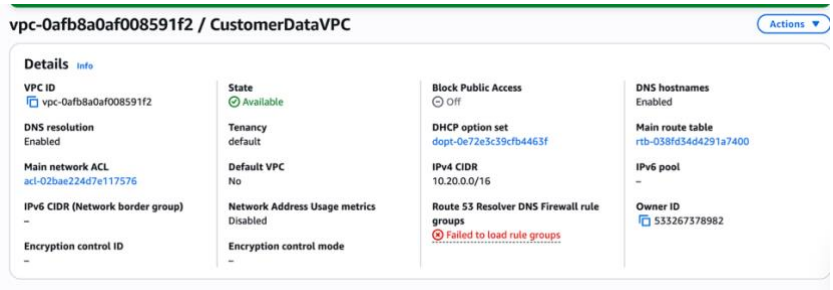

Figure: 2 VPC Configuration

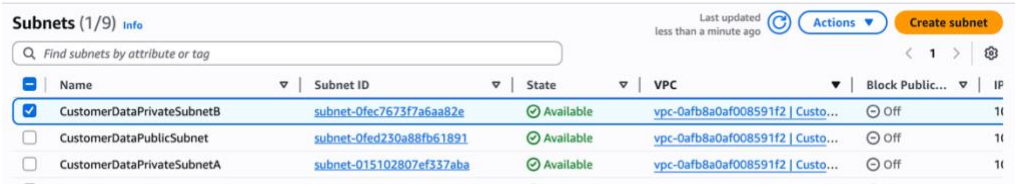

Figure: 3 Subnets Configuration

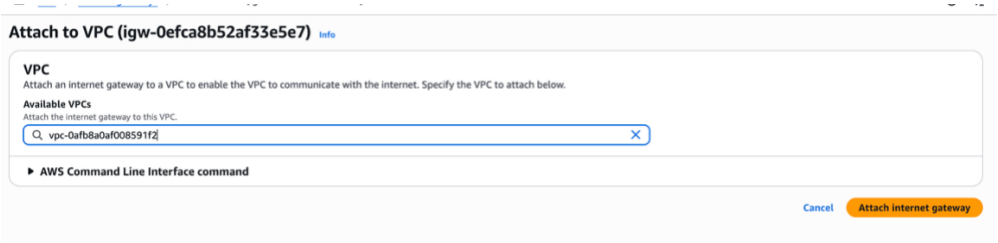

Figure 4: Internet Gateway attachment

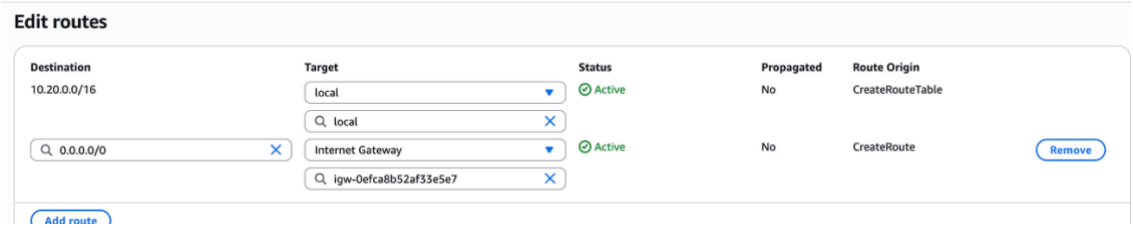

Figure 5: Route table configuration

Storage Configuration

An S3 bucket was created to store customer data files. The functionality was validated by uploading and listing a sample CSV file.

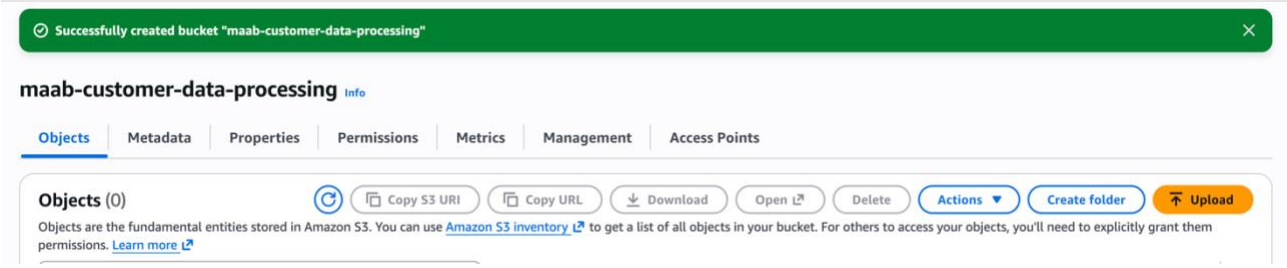

Figure 6: S3 Bucket

```
eee_W_6011404@runweb219702:~$ cat > customers.csv << 'EOF'
> id,name,email
> 1,Ali Khan,ali@example.com
> 2,Sara Ahmed,sara@example.com
> 3,Omar Hassan,omar@example.com
> EOF
eee_W_6011404@runweb219702:~$ aws s3 cp customers.csv s3://maab-customer-data-processing/
upload: ./customers.csv to s3://maab-customer-data-processing/customers.csv
eee_W_6011404@runweb219702:~$ aws s3 ls s3://maab-customer-data-processing/
2026-03-31 08:55:28        102 customers.csv
eee_W_6011404@runweb219702:~$
```

Figure 7: S3 file upload validation

Security Configuration

Security groups were configured to control access between system components. The RDS instance allows database access only from the EC2 security group.

Additionally, the principle of least privilege should be applied when configuring IAM roles and security groups to minimize unnecessary access.

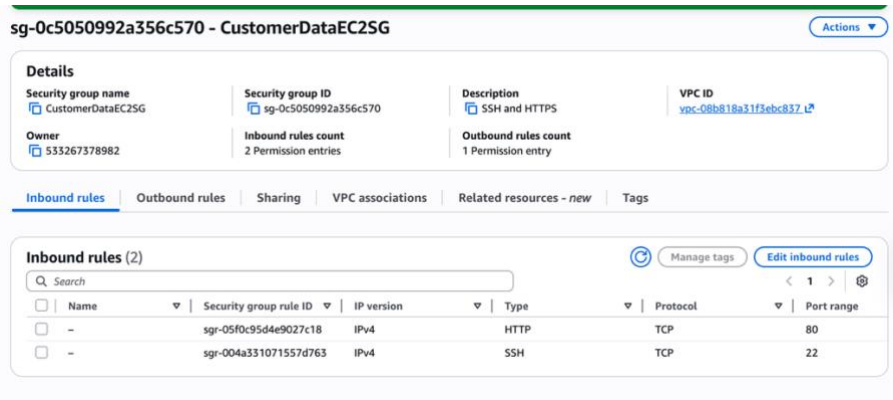

Figure 8: EC2 security group

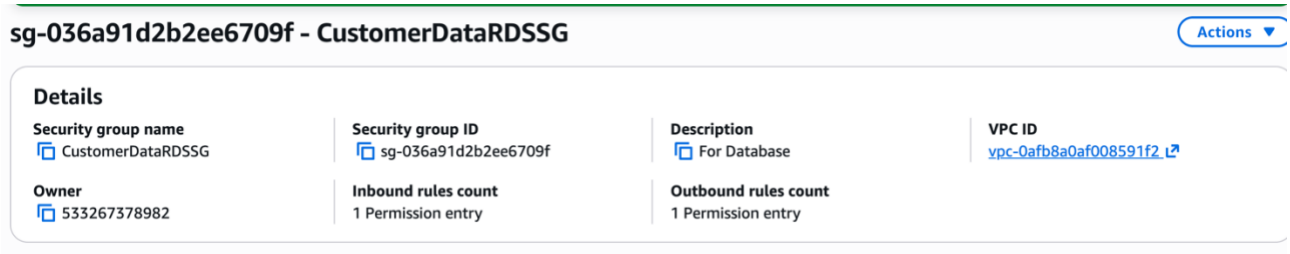

Figure 9 RDS Security Group

Compute and Database Deployment

An EC2 instance was deployed in the public subnet to act as the processing layer. The RDS database was deployed in private subnets using a DB subnet group.

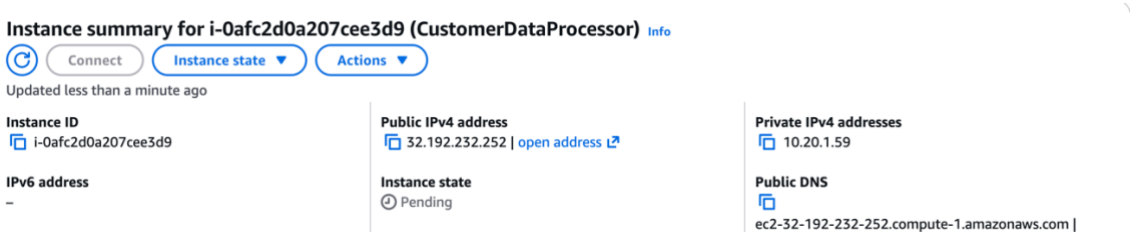

Figure 10: EC2 instance

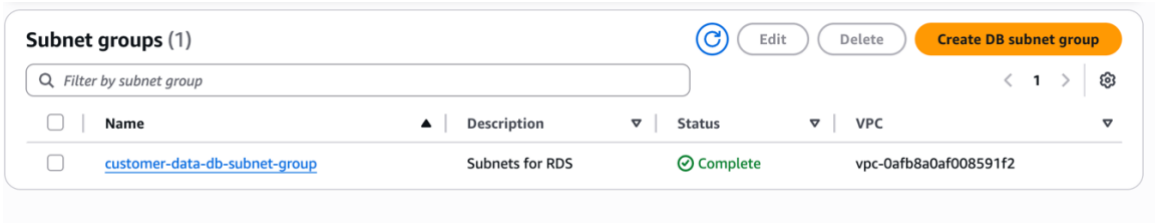

Figure 11: DB subnet group

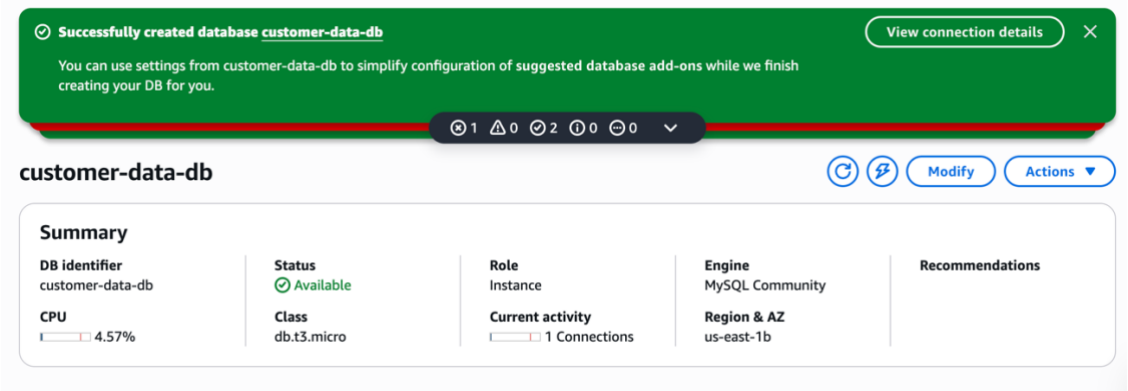

Figure 12: RDS Instance

Implementation Steps

The implementation started by creating the networking environment, including the VPC, subnets, Internet Gateway, and route table. The public subnet was configured to allow internet access through the Internet Gateway.

Next, security groups were created to control traffic between components. The EC2 security group allows SSH access, while the RDS security group allows database access only from the EC2 security group.

An EC2 instance was launched in the public subnet to act as the processing layer. A DB subnet group was created using the private subnets, and an RDS instance was deployed within those subnets to ensure it remains private.

An S3 bucket was also created to store sample customer data files. The system components were then validated through practical testing.

The S3 functionality was validated using the lab environment terminal, which has preconfigured AWS CLI credentials. A sample customer data file was successfully uploaded to and retrieved from the S3 bucket, confirming that object storage is functioning correctly. In a production environment, access to S3 would be managed through IAM roles attached to EC2 instances instead of relying on preconfigured credentials.

Problems and Solutions

During the implementation, an issue was encountered when attempting to install the MySQL client on the EC2 instance using the yum package manager. The installation failed because the instance was running Amazon Linux 2023, which does not include the mysql package in the default repositories.

This issue was diagnosed based on the error message returned by the package manager. The problem was resolved by installing a MariaDB compatible client instead, which provides the required MySQL functionality.

This experience highlighted the importance of understanding differences between Linux distributions and adapting commands accordingly.

Validation

The system was validated through SSH access, S3 interaction, and database connectivity testing.

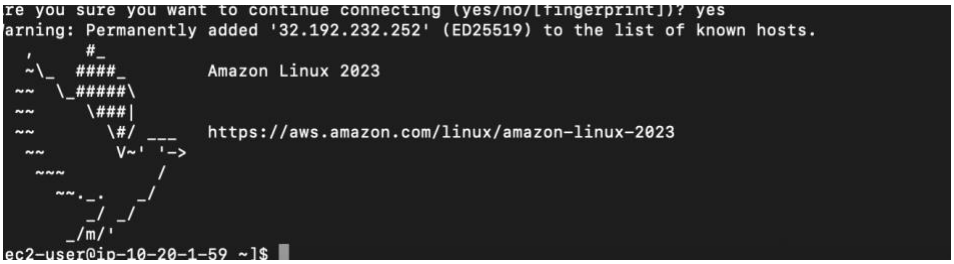

Figure 13 SSH connection to EC2

Next, the EC2 instance was used to connect to the RDS database using the MySQL client. A test database and a customers table were created. Sample customer records were inserted into the table, and a query was executed to retrieve the data successfully.

```
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 48
Server version: 8.4.7 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> CREATE DATABASE customer_db;
Query OK, 1 row affected (0.133 sec)

MySQL [(none)]> USE customer_db;
Database changed
MySQL [customer_db]>
MySQL [customer_db]> CREATE TABLE customers (
    ->     customer_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     full_name VARCHAR(100),
    ->     email VARCHAR(100),
    ->     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    -> );
Query OK, 0 rows affected (0.167 sec)

MySQL [customer_db]>
MySQL [customer_db]> INSERT INTO customers (full_name, email)
    -> VALUES
    -> ('Ali Khan', 'ali@example.com'),
    -> ('Sara Ahmed', 'sara@example.com'),
    -> ('Omar Hassan', 'omar@example.com');
Query OK, 3 rows affected (0.033 sec)
Records: 3  Duplicates: 0  Warnings: 0

MySQL [customer_db]>
MySQL [customer_db]> SELECT * FROM customers;
```

Figure 14 Database connection

```
MySQL [customer_db]>
MySQL [customer_db]> SELECT * FROM customers;
+-------------+-------------+-----------------+---------------------+
| customer_id | full_name   | email           | created_at          |
+-------------+-------------+-----------------+---------------------+
|           1 | Ali Khan    | ali@example.com | 2026-03-31 16:07:59 |
|           2 | Sara Ahmed  | sara@example.com | 2026-03-31 16:07:59 |
|           3 | Omar Hassan | omar@example.com | 2026-03-31 16:07:59 |
+-------------+-------------+-----------------+---------------------+
3 rows in set (0.001 sec)
```

Figure 15 Query execution

These tests confirmed that the EC2 instance can communicate with both the S3 bucket and the RDS database, and that the architecture functions as intended.

Enhancements

The current architecture can be improved by adding monitoring, logging, and alerting capabilities. CloudWatch can be used to monitor EC2 and RDS performance, while SNS can provide notifications for system events. Security can be enhanced by using IAM roles and stricter access policies. Backup strategies and automated scaling could also be introduced to improve reliability and performance.

Conclusions

This project successfully demonstrated the deployment of a customer data processing system on AWS. The architecture provides a solid foundation for further enhancements in

Phase 2. The implementation showed how different AWS services can be integrated to form a functional and secure backend system. Overall, the project achieved its goals and provided valuable hands-on experience in cloud infrastructure design and validation.

Appendix

Appendix A: CLI Commands Used

The following AWS CLI and system commands were used during the implementation:

- SSH connection to EC2:

```
ssh -i labsuser.pem ec2-user@<public-ip>
```

- Connect to RDS from EC2:

```
mysql -h <rds-endpoint> -u admin -p
```

- Create database and table:

```
CREATE DATABASE customer_data;
USE customer_data;
CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100)
);
```

- Insert sample data:

```
INSERT INTO customers (name, email)
VALUES ('Alice', 'alice@email.com'),
       ('Bob', 'bob@email.com');
```

- Query data:

```
SELECT * FROM customers;
```

- Upload file to S3:

```
aws s3 cp customer.csv s3://<bucket-name>/
```

- List files in S3:

```
aws s3 ls s3://<bucket-name>/
```

These commands demonstrate the interaction between EC2, RDS, and S3 services.

Appendix B: GitHub Repository

The full implementation, including CLI commands and documentation, is available in the GitHub repository: https://github.com/maab-osman/customer-data-processing-system-aws

Appendix C: Architecture Diagram

The architecture diagram was created using draw.io and represents the system components and their relationships within the AWS environment.

Appendix D: Screenshots

Screenshots included in the report provide evidence of:

- VPC and subnet configuration
- Internet Gateway and route table setup
- Security group rules
- EC2 and RDS deployment
- S3 storage and file upload
- Successful SSH and database connection
- Query execution results

These screenshots validate that the system was successfully implemented and tested.

References

Amazon Web Services Documentation. (2026). Amazon VPC User Guide.

https://docs.aws.amazon.com/vpc/

Amazon Web Services Documentation. (2026). Amazon EC2 User Guide.

https://docs.aws.amazon.com/ec2/

Amazon Web Services Documentation. (2026). Amazon RDS User Guide.

https://docs.aws.amazon.com/rds/

Amazon Web Services Documentation. (2026). Amazon S3 User Guide.

https://docs.aws.amazon.com/s3/

Amazon Web Services Documentation. (2026). AWS CLI Command Reference.

https://docs.aws.amazon.com/cli/

AWS Academy Learner Lab Instructions. (2026). Course materials and lab guidelines.

Additional troubleshooting and implementation insights were based on practical experimentation within the AWS Learner Lab environment.